

Proyecto Web — Cesde Courses Web

1. Problema y solución

- ▶ Problema: administrar cursos y docentes con filtros, validación y CRUD sin backend real.
- ▶ Solución: React + TypeScript + Vite + json-server.
- ▶ Resultado: experiencia de uso clara y funcional.
- ▶ Pregunta: ¿la interfaz representa bien la lógica del negocio?

2. Qué se construyó

- ▶ SPA con navegación por cursos y docentes.
- ▶ CRUD completo con modal y confirmación de eliminación.
- ▶ Filtros por texto, docente, precio, fecha y duración.
- ▶ Validación visual y bloqueo de formulario inválido.
- ▶ Responsive para móvil, tablet y escritorio.

3. Arquitectura de la web

- ▶ App: enrutamiento y layout global.
- ▶ Features: lógica por dominio.
- ▶ Shared: tipos, API client y validación reusable.
- ▶ UI: componentes visuales y estilos base.
- ▶ Flujo: componente -> hook -> cliente -> estado -> render.
- ▶ Error: mezclar red y vista en el mismo nivel.

4. Base de datos vs mock

- ▶ Base de datos relacional: tabla, fila y columna.
- ▶ SGBD: garantiza integridad y consultas.
- ▶ json-server: simula backend REST con JSON.
- ▶ Contraste: mock ayuda a prototipar, pero no reemplaza FK ni transacciones.
- ▶ Ejemplo: `curso.docente_id` en MySQL vs `docenteId` en JSON.
- ▶ Comprobación: la web puede simular la relación, pero no la protege.

5. Modelo, cardinalidad y restricciones

- ▶ Relación: Docente 1:N Curso.
- ▶ PK identifica cada registro.
- ▶ FK conecta entidades.
- ▶ UNIQUE evita duplicados.
- ▶ NOT NULL exige dato.
- ▶ Ejemplo: nombre de curso único, documento y correo únicos.
- ▶ Error: asumir que la validación UI es suficiente.
- ▶ Comprobación: el backend debe rechazar lo inválido.

6. Validación y reglas de negocio

- ▶ Validación en blur y submit.
- ▶ Curso: nombre, descripción, fecha, duración, precio y docente.
- ▶ Docente: nombre, documento y correo.
- ▶ El botón se deshabilita si el formulario es inválido.
- ▶ Error: confiar solo en la vista.
- ▶ Comprobación: un dato malo debe ser rechazado por la API.

7. Filtros y consultas

- ▶ `q` busca texto.
- ▶ `docenteId` filtra por docente.
- ▶ `precioMin`, `precioMax`, fechas y duración.
- ▶ `ORDER BY` y paginación simulan un backend real.
- ▶ El estado vive en la URL.
- ▶ Error: filtrar solo en UI sin reproducir la lógica del servidor.
- ▶ Comprobación: el mismo filtro debe mantenerse al recargar.

8. SQL, normalización y calidad del diseño

- ▶ SQL: SELECT, INSERT, UPDATE, DELETE.
- ▶ JOIN conecta tablas relacionadas.
- ▶ GROUP BY agrupa resultados.
- ▶ Normalización: 1FN, 2FN y 3FN.
- ▶ Ejemplo: separar docente y curso evita repetir nombre del docente.
- ▶ Error: usar INNER JOIN cuando importa conservar registros sin relación.
- ▶ Comprobación: el resultado refleja la semántica del negocio.

9. Calidad, despliegue y límites

- ▶ `tsc --noEmit` valida tipos.
- ▶ ESLint exige calidad y consistencia.
- ▶ Build con Vite para producción.
- ▶ GitHub Pages comparte la SPA.
- ▶ Límite: la demo pública no persiste cambios reales.
- ▶ Error: creer que la UI pública es infraestructura completa.
- ▶ Comprobación: CRUD real requiere levantar el proyecto localmente.

10. Cierre ejecutivo

- ▶ La web demuestra UX, flujo y validación.
- ▶ Pero la integridad real se define en la base de datos y en la API.
- ▶ La calidad no es estética; es consistencia, estructura y comprobación.
- ▶ Cierre: una interfaz útil solo vale si el dato se sostiene.